ZenQL

Design Document

Ansh Gupta

M.Tech, Computer Science & Engineering

Indian Institute of Technology Kharagpur

Student ID: 25CS60R34

GitHub Repository:

https://github.com/AnshGupta01/ZenQL

March 2026

This document describes the current architecture and behavior of ZenQL as implemented in the repository above.

Contents

1 System Overview **2**
1.1 Core Runtime Path 2

2 Server and Concurrency Model **2**
2.1 Connection-Level Concurrency 2
2.2 Database-Level Concurrency 2
2.3 Intra-Query Parallelism 2

3 SQL Subset and Parsing **2**
3.1 Implemented Query Types 3
3.2 Parser Behavior Notes 3

4 Storage Architecture **3**

5 Indexing Method **3**
5.1 FastHashIndex 3
5.2 BTree Scaffolding 4

6 Caching Strategy **4**
6.1 Thread-Local Table Cache (`TableCache`) 4
6.2 Versioned Join Hash Cache (`JoinHashCache`) 4

7 Query Execution Path **4**
7.1 Select Execution 4
7.2 Join Execution 4
7.3 Delete Execution 4
7.4 Result Serialization 5

8 TTL and Row Expiration **5**

9 Persistence and Recovery **5**
9.1 Checkpoint Lifecycle 5

10 Network Protocol **5**

11 Build and Execution **5**
11.1 Build 6
11.2 Run Server 6
11.3 Run REPL (Client) 6
11.4 Run Benchmarks 6

12 Performance Results **6**

13 Current Limitations **6**

14 Design Intent **7**

1 System Overview

ZenQL is a **TCP client/server SQL-like engine** written in **C++17**. It implements a compact relational query processor with in-memory columnar storage, hash indexing, and a plaintext wire protocol, designed for high-throughput benchmarking of parser, index, and concurrency strategies.

1.1 Core Runtime Path

The end-to-end execution sequence is:

1. Client calls `flexql_exec` in `src/client/libflexql.cpp`.
2. SQL is transmitted to the server as newline-terminated text.
3. Server in `src/server/server.cpp` reads lines, executes each statement through `OptimizedDatabase`, and sends text output back.
4. Client reads the response until the sentinel `\x00END\x00`, then parses tab-delimited rows and invokes a callback per row.

2 Server and Concurrency Model

The concurrency model operates at two independent layers.

2.1 Connection-Level Concurrency

- The server accepts sockets and dispatches each client connection to a shared `ThreadPool`.
- `TCP_NODELAY` is enabled to minimize interactive latency.

2.2 Database-Level Concurrency

- `OptimizedDatabase` uses a global `std::shared_mutex` around table and index maps.
- Read-heavy operations acquire *shared* locks.
- Structural writes (e.g., `CREATE TABLE`) acquire *unique* locks.

2.3 Intra-Query Parallelism

Within `INNER JOIN`, equal-key joins are parallelized by splitting the probe side into chunks and dispatching them across worker threads in the pool.

3 SQL Subset and Parsing

The parser lives in `src/parser/parser.cpp` and produces `ParsedQuery` variant objects as defined in `src/parser/parser.h`.

3.1 Implemented Query Types

- `CREATE TABLE [IF NOT EXISTS]`
- `INSERT INTO ... VALUES (...)` – single or batched rows
- `SELECT ... FROM ...`
- Optional `WHERE` with comparison operators: `=`, `>`, `<`, `>=`, `<=`
- Optional `ORDER BY <column> [DESC]`
- `INNER JOIN ... ON ...`
- `DELETE FROM <table> [WHERE ...]`

3.2 Parser Behavior Notes

- Table and column identifiers are validated to alphanumeric characters plus underscore.
- SQL keywords are case-insensitive.
- Semicolon is optional; trailing `;` is stripped during parse.
- `CHECKPOINT` is recognized by the parser but is not currently dispatched as a dedicated execution branch in `OptimizedDatabase`.

4 Storage Architecture

ZenQL uses a **Columnar Storage Model** (Decomposed Storage) for high-performance in-memory operations.

- **Columnar Vectors:** Each table consists of a collection of `StringColumn` instances. Each column's data is stored in a contiguous `std::vector<std::string>`, improving cache locality during scans and filters.
- **Expiry Metadata:** Row expiration timestamps are stored in a separate parallel vector, enabling rapid liveness checks without touching row data.
- **Memory Management:** Rows are imported into the underlying vectors using `std::move` where possible to minimize string copies.

5 Indexing Method

ZenQL maintains primary key lookups via a **Fast Hash Index**.

5.1 FastHashIndex

- Provides $O(1)$ point lookup acceleration for primary key columns.
- The primary key column index is derived from the schema (`PRIMARY KEY` declaration); if no primary key is declared, the index defaults to column 0.

5.2 BTree Scaffolding

`BTreeIndex` scaffolding exists in `src/index/btree_index.h` and is reserved for future range query optimizations. It is not currently active in the execution path.

6 Caching Strategy

Two primary caching layers reduce lock contention and redundant computation.

6.1 Thread-Local Table Cache (`TableCache`)

Each thread maintains a local cache of the most recently accessed table's pointer, primary index, and precomputed result headers. This avoids global map lookups and shared mutex acquisitions for repeated operations on the same table.

6.2 Versioned Join Hash Cache (`JoinHashCache`)

- For `INNER JOIN` operations, the probe-side hash table is cached globally.
- Each cache entry is associated with a **Table Version Number**.
- On any `INSERT` or `DELETE`, the table version increments and existing join caches for that table are invalidated.
- This ensures repeated joins over a large table (e.g., 10M rows) remain fast after the initial hash-table build.

7 Query Execution Path

`OptimizedDatabase::execute_to_buffer` in `src/query/optimized_database.h` is the central execution entry point.

7.1 Select Execution

- **Fast path:** `SELECT * FROM <table> WHERE <pk> = <value>` uses a cached table pointer and hash index for a direct $O(1)$ lookup.
- **General path:** Full scan with optional filter predicate and `ORDER BY` sorting; supports both `*` projection and named column subsets.

7.2 Join Execution

- Hash-assisted equality joins for `ON <col> = <col>` predicates.
- Non-equality predicates fall back to a nested comparison loop.
- Optional post-join `WHERE` filter and `ORDER BY` are applied after the join phase.

7.3 Delete Execution

Supports full-table delete or conditional delete. Retained rows are rebuilt into a fresh column store and the hash index is reconstructed.

7.4 Result Serialization

All results are serialized to a string buffer as tab-delimited rows with a status/header prefix, then flushed over the TCP socket.

8 TTL and Row Expiration

Rows carry separate expiration metadata in the table storage layer.

- `INSERT ... EXPIRY <seconds>` computes an absolute expiry timestamp as $t_{\text{now}} + \Delta t_{\text{seconds}}$.
- Expired rows are skipped silently during `SELECT` and `JOIN` lookups.
- Expired rows are *not* immediately compacted; physical removal occurs only on the next delete/rewrite pass over that table.

9 Persistence and Recovery

Persistence is enabled at server startup via:

```
OptimizedDatabase global_db("data", true);
```

9.1 Checkpoint Lifecycle

- **Startup:** `CheckpointManager` attempts recovery from the `data/` directory.
- **Shutdown:** On `SIGINT` or `SIGTERM`, the server triggers `save_checkpoint()` before exiting.

This provides crash-restart durability at checkpoint granularity, with no WAL or fine-grained transaction logging.

10 Network Protocol

The wire protocol is intentionally plaintext and line-based for simplicity and low overhead.

Direction	Format
Request	Newline-separated SQL statements (`\n` delimited)
Response	Text terminated by sentinel `\x00END\x00`
Row format	Tab-separated column values

The protocol requires no framing library and is fully readable with standard TCP tools such as `nc` or `telnet`.

11 Build and Execution

ZenQL is built with standard C++17 tools and has **no external runtime dependencies**.

11.1 Build

```
make clean && make
```

11.2 Run Server

```
make run-server
```

11.3 Run REPL (Client)

```
make run-repl
```

11.4 Run Benchmarks

```
# General insertion benchmark
make run-benchmark ARGS="10000000"

# Direct binary execution
./bin/benchmark_flexql 10000000

# Run internal unit tests
./bin/benchmark_flexql --unit-test
```

12 Performance Results

The following results were measured on 10 million rows.

Metric	Value
Environment	MacBook Air M1, 8 GB RAM
Dataset size	10,000,000 rows
Elapsed time	21,651 ms
Throughput	≈461,872 rows/sec
Unit test correctness	22/22 passed (100%)

13 Current Limitations

The following features are not yet implemented:

- **Transactions:** `BEGIN`, `COMMIT`, and `ROLLBACK` are absent; all operations are auto-committed.
- **Update command:** No `UPDATE ... SET ...` statement.
- **Multi-statement transactions:** Beyond newline-delimited independent statements, there is no logical grouping of statements.
- **Error diagnostics:** Error messages are concise and not yet structured (e.g., no error codes or source positions).

- **Secondary indexes:** `BTreeIndex` scaffolding exists but secondary index utilization in execution paths is limited.

14 Design Intent

ZenQL was designed with three explicit priorities:

1. **Fast local development cycle** – single `Makefile`, no external runtime dependencies.
2. **High-throughput in-memory operations** with practical persistence checkpoints for restart durability.
3. **Compact SQL subset** suitable for benchmarking parser throughput, index efficiency, and concurrency strategies under load.